

The Combined Agent-Memory Stack

How TDAI, Falda, and UMP work together to give long-lived AI agents durable, cross-agent memory

White paper — Rick Stevens (Argonne National Laboratory). Compiled by Ollie, 2026-06-25. Companion to: ARCHITECTURE_BRIEF.md (ground truth), SYSTEM-SPEC-1000-AGENTS.md (engineering spec), the TDAI/Falda decks, and the UMP reference repo.

Executive summary

We operate two AI agents — **Ollie** (on CherryRd) and **Kukla** (on an M1 Mac mini) — that must remember things across sessions, machines, and chat surfaces. A single chat window has no memory of the others; without an explicit memory layer the default behavior is to re-derive completed work from scratch. The combined stack solves this with three cooperating layers:

1. **TDAI** — the **live production memory runtime**. A four-layer (L0->L1->L2->L3) local capture/distill/recall pipeline running as an OpenClaw plugin on each host. It is the only system currently in the agents' live prompt path.
2. **Falda** — a **born-clean, US-origin standalone** re-implementation of the same four-layer design, hardened for multi-tenant scale (1,000 agents). It currently runs in **shadow mode** alongside TDAI (evidence-gated, reversible) and is the planned successor.
3. **UMP** — a lightweight, **schema-first cross-agent reference store** (JSONL + CLI + HTTP) used as an additional durable layer for facts both agents must share, and as the design reference for record schemas and memory policy.

The design principle throughout: **memory augments durable file-backed truth; it never replaces it**. MEMORY.md, daily notes, TOOLS.md, keychain/1Password secrets, and mailbox context remain sources of truth. The stack adds fast semantic recall and cross-agent aggregation on top.

1. The problem

Each chat surface (Telegram DM, Slack, the bridge group, the OpenClaw CLI, webhooks) is a **separate session with zero in-session memory of the others**. Agents also wake “fresh” each session. Three concrete failure modes motivated the stack:

- **Amnesia / re-derivation**: an agent re-does completed work because the prior result lived in a different session.
- **Cross-agent blindness**: Ollie cannot see what Kukla learned, and vice versa, unless it is written somewhere both can read.
- **Provenance drift**: a fact written by one agent into a shared file can be silently overwritten or fabricated; each agent needs to verify, not trust, peer-written provenance.

The mandatory recall discipline (search memory **before** answering any status/recall/config question; label findings *remembered* / *live-verified* / *assumed* / *unknown*) is the behavioral counterpart to the technical stack.

2. The four-layer model (shared by TDAI and Falda)

Both TDAI and Falda implement the same conceptual pipeline. Raw conversation flows in at L0 and is progressively distilled into higher-value, lower-volume representations:

Layer	Name	What it holds	How it is produced
L0	Conversation recording	Every raw turn (user/assistant/system/tool)	Auto-captured on each turn; dual-written to the store + JSONL daily shards
L1	Memory extraction	Structured memories: persona / episodic / instruction	An LLM extracts from L0; vector dedup + conflict detection
L2	Scene induction	Topic-clustered Markdown “scene blocks”	An LLM incrementally induces/updates scenes from L1

Layer	Name	What it holds	How it is produced
		with a heat score	
L3	Persona	A long-form profile of the user/agent	An LLM regenerates from the scene blocks

Runtime hooks (TDAI, live): - **Auto-recall** on `before_prompt_build`: hybrid-searches relevant memories + loads persona, injects them into the system context. (Requires prompt-injection to be enabled; if disabled, recall silently no-ops — a known safety control.) - **Auto-capture** on conversation end: records L0, then a pipeline scheduler triggers L1->L2->L3 after N conversations or an idle timeout.

Retrieval is RRF-hybrid: sparse BM25/keyword fused with dense vector similarity, with a pure-vector and a keyword-only mode available. Embeddings are 1536-dim (`text-embedding-3-small`, served locally via the Argo proxy — free, on-prem, no external dependency).

3. Layer A — TDAI (the live runtime)

What it is: an OpenClaw plugin (`@tencentdb-agent-memory/memory-tencentdb`, MIT) running on both hosts. It is the **only system in the agents' live prompt path today**.

Storage layout (`~/openclaw/memory-tdai/` on CherryRd): - `conversations/` — L0 daily JSONL - `records/` — L1 daily JSONL - `scene_blocks/*.md` — L2 scenes (e.g. AI-Benchmark-Operations, Paper-Replication-Project, Falda-Memory-System-Stewardship) - `vectors.db` — SQLite + `sqlite-vec` vector DB (~46 MB live) - `persona.md` — L3 persona (~42 KB live) - `.metadata/` (manifest + checkpoints), `.backup/` (rolling)

Backends (pluggable): `sqlite` (default, fully local — SQLite + `sqlite-vec` + local or proxied embeddings) or `tcvdb` (Tencent Cloud Vector DB). **In production we run the local sqlite backend**; no Tencent Cloud egress occurs at runtime, and the LLM/embedding calls go to the free on-prem Argo/ALCF endpoints.

Multi-tenant status: TDAI's local plugin has been run multi-store (`maxStores~1000`, several live instances), but it is fundamentally **single-tenant per store** (one `dbPath` / blob dir). True fleet-scale multi-tenancy is a Falda concern (§4, §5).

Why replace it: the package is Chinese-origin (authored by the “TencentDB Agent Memory Team,” Chinese-dominant README, jieba tokenizer, optional Tencent COS/TCVDB). For a US-government-adjacent deployment we want a **born-clean US-origin** equivalent — hence Falda. The coupling is narrow (~2 Tencent npm packages + ~6 source files behind a single IMemoryStore interface + the jieba tokenizer), so the rebuild is adapter-and-deploy work, not a ground-up rewrite.

4. Layer B — Falda (the born-clean successor)

What it is: a public, Apache-2.0, **clean-room US-origin standalone** of the same four-layer system — *not* a fork carrying history. Repo: rick-stevens-ai/falda. Ollie is steward-of-record (his full-scope classic PAT has push authority; he also validates origin-clean on Kukla’s commits).

Stack: TypeScript gateway over **SQLite + sqlite-vec + FTS5**, RRF-hybrid retrieval, a distiller that promotes L0->L3, and a tap that feeds it. better-sqlite3 was bumped 11->12 for Node 26 prebuilds; 13/13 smoke green.

Born-clean invariant (hard rule): the public repo’s prose/commits contain **zero** references to tencent/tdai/alibaba/repatriation/china/tcvdb/jieba (git grep == 0). This constrains only the public repo, not internal memory/diary notes.

Multi-tenant design (approved): Falda adds two-axis (tenant, pool) addressing on every operation, with **store-per-(tenant,pool) physical isolation** (each tenant+pool = its own SQLite file + blob dir) chosen over column-tagged filtering. tenant = required agent identity; default private pool = self (TDAI parity); opt-in named pools allow sharing with per-member access modes (none / read / readwrite). Strict isolation; scoped single-target recall.

Shadow-mode dual-run (Rick-approved, one week): Falda runs alongside TDAI on **both hosts as two independent deployments** — Kukla/m1 (launchd gateway + tap, 560-turn backfill) and Ollie/CherryRd (gateway on :8077, independent DB, Argo embeddings, seeded to TDAI parity: ~4,231 L0->Stream, ~554 L1->Atoms, 7 scenes, persona->Core). **TDAI remains the only live production runtime** throughout. A weekly parity/cutover cron (fires 2026-06-29) reports TDAI-vs-Falda parity plus distiller tier output from both hosts as the cutover decision input.

Migration rule (hard): TDAI->Falda cutover must be a **planned restart, never a hot-swap**. The shadow dual-run is a deliberately reversible, evidence-gated experiment on isolated ports/DBs; TDAI stays canonical until a deliberate cutover.

5. Scaling to 1,000 agents (the engineering spec)

The `SYSTEM-SPEC-1000-AGENTS.md` defines the buildable target. Key points:

Agent-facing contract (preserved tool names): `capture`, `recall`, `memory_search` (-> L1), `conversation_search` (-> L0), `read` (scene/persona), plus the UMP compat surface. A new **scope parameter** (`self` / `cohort` / `all`) governs private vs. swarm recall.

Three-tier architecture: **ingest** (durable L0 capture onto a partitioned bus) -> **distill** (horizontal extraction worker pool, L1->L2->L3, idempotent) -> **aggregate** (a swarm aggregator that dedups, conflict-resolves via multi-judge, and rolls up cross-agent knowledge into a shared swarm tier). A control plane holds tenancy, quotas, and checkpoints.

Locked technology choices (Rick-approved): Qdrant (vector store), NATS JetStream (bus, already run for Sibline), Tantivy (embedded BM25), CELS compute nodes for store/bus/workers with GPU boxes reserved for embeddings + LLM extraction.

The real ceiling is LLM-extraction throughput, not storage. At 1,000 agents x ~200 turns/day the storage and embedding load is trivial (~700 MB/day raw; one A100 embedder is >10k/s vs. a ~1.4/s requirement). The bottleneck is the rate at which LLMs can distill L0->L1.

Acceptance gates (Definition of Done): 1,000 concurrent writers with capture-ack <10 ms and zero loss; recall p99 <100–150 ms; any agent's memory fleet-queryable in <30 s; federated cross-agent search returns merged+deduped results; pipeline holds steady state; graceful degradation to per-agent self-recall if the aggregation tier is lost; a published `{nodesxpartitionsxworkers}`->agent-count table; and a clean SBOM (US/allied origin only).

6. Layer C — UMP (cross-agent reference store)

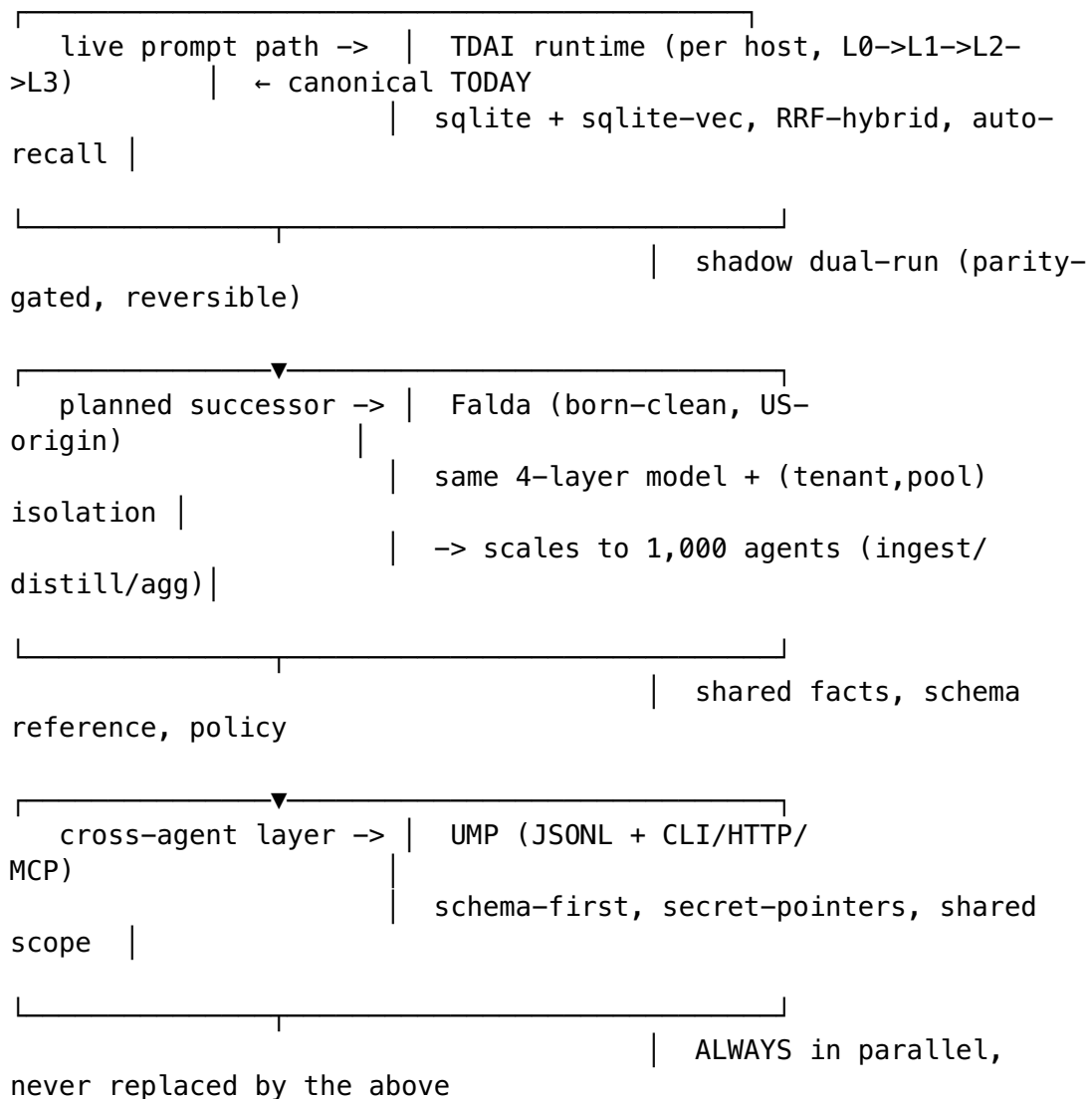
What it is: a lightweight, schema-first JSONL memory store with a CLI, HTTP API, and MCP surface (`ump-recall`, `ump-get`, `ump-capabilities`). Repo: `rick-stevens-ai/ump-memory` (public). It is the **additional durable cross-agent layer** for facts both agents must share.

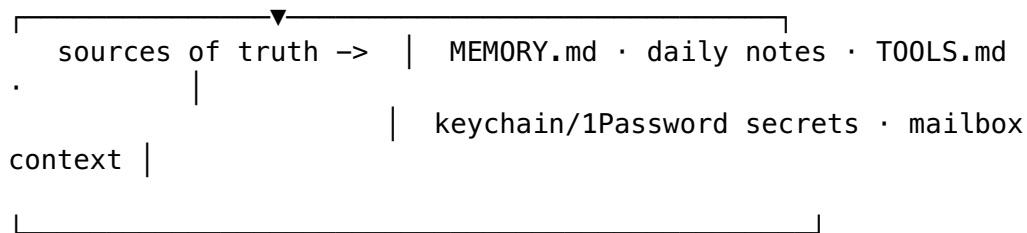
Record schema (the closest thing to a canonical cross-agent memory contract): kind in {semantic, episodic, procedural, working, identity}; scope carries owner / agent / project / visibility (private|shared|public); plus tags, source (binding/path/line), salience, and timestamps.

Secret policy (enforced): never store raw keys/tokens — store a **pointer** (e.g. “key is in macOS Keychain service X, account Y; use env var Z”). This mirrors the stack-wide rule that secrets live in keychain/1Password/env-files, never in memory or chat.

Standing policy: durable cross-agent facts go into UMP as an **additional layer — never only there**. File-backed sources of truth (MEMORY.md, TOOLS.md, daily notes) are always kept in parallel. UMP augments; it does not replace.

7. How the layers fit together





- **Per-agent depth** comes from TDAI today (and Falda tomorrow): each agent's own L0->L3.
- **Cross-agent breadth** comes from UMP today and the Falda **swarm tier** at scale: the cohort/all scope and the aggregator make one agent's knowledge queryable by the fleet.
- **Verifiability** comes from the file-backed bottom layer: anything important is also on disk in a human-readable form an agent can re-check rather than trust.

Embedding & inference are unified and free: all three layers use the on-prem Argo proxy (text-embedding-3-small, 1536-dim) and free Argo/ALCF/CELS endpoints for extraction — no paid or foreign dependency in the memory path.

8. Current state (2026-06-25)

Component	State
TDAI runtime (CherryRd + m1)	LIVE / canonical. sqlite backend, ~46 MB vectors.db, ~42 KB persona, 6+ scene blocks per host.
Falda repo	Public (Apache-2.0), born-clean (git grep tdai==0), 13/13 smoke; Ollie steward; multi-tenant (tenant,pool) design approved.
Falda shadow dual-run	Running on both hosts (m1 launchd gateway+tap+distiller; CherryRd gateway :8077), seeded to TDAI parity, capturing. Reversible.
Parity/cutover gate	Weekly cron fires 2026-06-29 09:00 with TDAI-vs-Falda parity + distiller tiers as cutover input.
1,000-agent spec	Complete; technology choices locked; next step is Phase-0 stand-up.
UMP	

Component	State
	Public reference repo live (schema, prompts, docs, secret-pointer policy).

Open items: finalize the distiller split (Ollie owns repo + distiller.ts; Kukla wires the tap); resolve the strict-born-clean vs. allow-integrations/tdai/policy question on the public repo; execute the planned-restart cutover only after the 2026-06-29 parity evidence.

9. Design principles (the through-line)

- **Memory augments file-backed truth; it never replaces it.**
- **Free, on-prem inference only** in the memory path (Argo/ALCF/CELS).
- **Verify peer-written provenance; don't trust it** — own your ledger.
- **Planned restart, not hot-swap**, for any runtime migration.
- **Born-clean for anything public**; internal notes are unconstrained.
- **Secrets are pointers, never values.**
- **The ceiling is LLM-extraction throughput, not storage** — design the distill tier first.

Artifacts referenced: ARCHITECTURE_BRIEF.md · SYSTEM-SPEC-1000-AGENTS.md · SCALE-1000-AGENTS-PLAN.md · US-REBUILD-MIGRATION-PLAN.md · Falda decks (Falda.pptx / Falda_System / Falda_Scaling) · tdai-memory-system.pptx · ump-memory/docs/record-schema.md.